

实验5 基于鲲鹏虚拟化的大模型部署与测试 实验报告

uu-matter543

2025-2026-1

1 调研部分

1.1 libvirt

1.1.1 libvirt 的背景

Libvirt 诞生于 2005 年，由 Red Hat 的工程师团队主导开发。其产生背景源于当时虚拟化技术的快速发展，但各类虚拟化平台缺乏统一的管理接口，导致管理工具碎片化、兼容性差的问题日益突出。为解决这一痛点，Libvirt 项目应运而生，旨在构建一个跨虚拟化技术的抽象管理层。

经过十余年的发展，Libvirt 已演变为支持多种虚拟化技术的综合管理平台，并成为众多云计算平台和桌面虚拟化工具的底层管理标准，在 Linux 虚拟化生态中占据了核心地位。

1.1.2 libvirt 的用途

Libvirt 的主要目标是提供一种跨虚拟化技术的通用管理接口，实现：

- 虚拟机生命周期管理：创建、启动、暂停、重启、关闭、迁移
- 虚拟资源管理：管理虚拟网络、存储池、虚拟磁盘、CPU/内存分配
- 监控与统计：获取虚拟机运行状态、性能指标
- 安全管理：资源隔离、访问控制
- 异构平台支持：统一管理不同虚拟化技术

1.1.3 基本工具集

Libvirt 采用客户端-服务器架构和模块化设计，主要组件包括：

- 核心服务层：
 - libvirtd 守护进程：运行在宿主机上的核心服务，处理所有管理请求
 - Hypervisor 驱动：适配不同虚拟化技术的插件模块
 - 状态管理机制：维护虚拟机状态的一致性和持久化
- 用户工具集：
 - virsh：命令行虚拟机管理工具
 - virt-manager：图形化桌面管理工具
 - virt-install：命令行虚拟机安装工具
 - virt-viewer：虚拟机控制台查看器
 - virt-clone：虚拟机克隆工具

- 开发接口层：
 - C 语言原生 API：提供完整的虚拟化管理功能
 - 多语言绑定：Python、Perl、Java、Go、Ruby等主流语言支持

1.1.4 虚拟机基本信息

- 基础配置：
 - 架构类型：ARM64 架构
 - 机器类型：QEMU 虚拟化平台
 - 虚拟化类型：完全硬件虚拟化
- CPU 与内存配置：
 - CPU 模式：直通宿主 CPU 特性
 - CPU 拓扑：2 Sockets × 2个Cores × 1个Thread = 4个vCPU
 - IO 线程：1 个专用 IO 线程 (iothreads=1)
- 共享文件夹：
 - 类型：mount（目录挂载）
 - 访问模式：passthrough（直通）
 - 宿主机路径：/var/lib/libvirt/shared/stu13/shared
 - 虚拟机内挂载点：host_share

1.1.5 使用方式

启动虚拟机：

```
virsh start openEulerVM-stu13
```

查看状态：

```
virsh list --all  
virsh dominfo openEulerVM-stu13
```

控制台访问：

```
virsh console openEulerVM-stu13
```

1.2 BitNet b1.58基本信息

1.2.1 产生背景

BitNet b1.58 是微软研究院于 2024 年提出的一种革命性的大语言模型架构，其核心贡献在于实现了从传统的“浮点计算范式”向“整数计算范式”的根本性转变。该架构的名称直接揭示了其最显著的特征：1.58 这一数字源于其模型权重仅采用三元值 $\{-1, 0, +1\}$ 进行表示。从信息论角度计算，存储一个具有三种状态的参数需要约 $\log_2 3$ 比特的存储空间，故得此名。

1.2.2 关键技术

BitNet b1.58 并未采用对预训练全精度模型进行后量化的常规路径，而是从头开始设计和训练。其前向传播过程主要包含三个关键步骤：

1. 对输入进行常规的线性投影；
2. 通过一个可微的“绝对值均值”量化函数，将权重矩阵实时、动态地量化为三元数值；
3. 核心的矩阵乘法运算转化为整数矩阵乘法，即使用低位宽的激活值与三元权重进行计算。模型的激活值仍保留相对较高的精度，以确保模型的表达能力和训练的稳定性。

1.2.3 优势

- 内存与带宽效率方面，三元权重使模型参数的存储空间相较于传统的 16 位浮点模型减少约 10 倍，同时极大地降低了内存访问带宽需求，有效缓解了现代硬件的主要性能瓶颈。
- 计算耗能及延迟方面，高度优化的整数运算远比浮点数运算高效，显著降低了推理过程的能耗和时延。
- 可扩展性方面，随着模型规模的扩大，其在语言建模任务上的性能能够逼近同尺寸的全精度 Transformer 模型，而其在能耗和速度方面的优势则愈发显著。

这些特性使得 BitNet b1.58 极具硬件友好性，特别适合在资源受限的边缘设备、移动终端及专用 AI 加速芯片上部署；还证明了三值化架构并非以牺牲大语言模型的性能为代价，而是为大型模型的高效推理开辟了一条全新的技术路径。

1.2.4 总结

BitNet b1.58 不仅仅是一种模型压缩技术，更代表了一种面向未来的高效 AI 计算范式。它通过极致的权重离散化，将模型计算转化为高效的整数运算，为实现绿色 AI、推动大语言模型在更广泛终端上的普惠应用奠定了坚实的基础。

2 实验目的

1. 掌握 libvirt 虚拟化工具的基本使用：包括虚拟机的创建、配置修改、启动、连接和关闭，理解 libvirt 在虚拟机管理中的作用。
2. 学习开源模型 BitNet b1.58 的部署与推理：通过实际运行模型推理和性能测试，了解该模型的特点、应用场景及性能表现。
3. 实践虚拟机与宿主机之间的文件共享：通过配置共享目录，实现宿主机与虚拟机之间的数据交互。
4. 分析模型推理效果与性能：通过修改 prompt 观察模型输出变化，通过 benchmark 测试分析模型在不同参数下的性能表现。

3 实验步骤

1. 进行准备工作：修改虚拟机名称、创建镜像目录、创建虚拟机与主机共享目录等。
2. 使用命令 `virsh start openEulerVM-stu13` 启动虚拟机，如图1所示。
3. 使用命令 `virsh edit openEulerVM-stu13` 修改虚拟机配置，如图2所示。
4. 修改完成后保存虚拟机配置，如图3所示。
5. 在虚拟机中使用命令 `conda --version` 验证 conda，如图4所示。
6. 在虚拟机中使用命令 `conda activate bitnet-cpp` 切换到虚拟环境，如图5所示。

7. 在虚拟机中使用如下命令进行推理，如图6所示：

```
python run_inference.py \  
-m models/BitNet-b1.58-2B-4T/ggml-model-i2_s.gguf \  
-p "You are a helpful assistant" -cnv
```

8. 继续推理的效果如图7所示。

9. 在虚拟机中使用如下命令进行 benchmark 测试，如图8所示。

```
python utils/e2e_benchmark.py \  
-m models/BitNet-b1.58-2B-4T/ggml-model-i2_s.gguf \  
-n 200 -p 256 -t 4
```

4 实验分析

在针对 BitNet-b1.58-2B-4T 模型的端到端基准测试中，使用命令

```
python utils/e2e_benchmark.py \  
-m models/BitNet-b1.58-2B-4T/ggml-model-i2_s.gguf \  
-n 200 -p 256 -t 4
```

运行测试，其中参数设定为：生成 token 数量 200 个，输入 prompt 长度 256 个 token，线程数 4 个。测试结果如下：

该模型为 BitNet-b1.58-2B-4T 的 $I2_s$ 版本，采用 2 位量化三元权重，参数量为 2.74B，量化后模型文件大小为 1.71 GiB。测试在 CPU 后端进行，批处理大小为 1，使用 4 线程。

性能测试分为两部分：一是提示处理，即对长度为 256 的输入序列的处理速度；二是 token 生成，即自回归生成 200 个 token 的速度。测试结果显示，提示处理速度为 8.92 tokens/秒，token 生成速度为 8.94 tokens/秒，两者性能基本一致，表明模型在输入处理和序列生成阶段的推理效率相近。

总体而言，该量化模型在 CPU 上以约 9 tokens/秒的速度运行，属于较低推理速度，适用于对实时性要求不高的离线测试或资源受限环境。

接下来，我又进行了对比实验：将 `-t` 处的参数改为 1 并运行测试，测试结果显示，提示处理速度为 2.41 tokens/秒，token 生成速度为 2.41 tokens/秒，相比 4 线程下降 73.0%，说明该模型能够利用 CPU 并发提高运行效率；理想情况下 4 线程执行效率应为 1 线程的 400%，而实际情况下效率为 370%，说明该模型在 CPU 并行化方面效率较高。

将 `-t` 处的参数改为 8 和 16 分别进行测试时，在 10 小时内都未得到测试结果，可能是由于为虚拟机分配的 CPU 数量为 4，在线程数过多时会产生大量的调度开销，导致无法产生结果；也可能超出了内存的带宽，导致所有线程都在等待内存访问，形成了内存墙（类似死锁）导致无法运行。

5 实验总结

在本实验中，我使用 libvirt 虚拟化工具进行了虚拟机的创建、配置修改、控制台访问等操作，运行了 BitNet 大语言模型观察了推理过程，并通过 benchmark 测试了大语言模型的性能，观察了单线程条件下的速度与多 CPU 并发的应用，收获颇丰。感谢课程组精心设计的虚拟化实验，也感谢老师和助教一直以来的付出和答疑解惑。

```

stu13@ascend07:~$ 
System information as of time: Sat Dec 20 04:35:09 PM CST 2025

System load: 20.24
Processes: 2056
Memory used: 2.0%
Swap used: 0.0%
Usage On: 64%
IP address: 111.186.61.17
IP address: 10.147.8.17
IP address: 192.168.122.1
IP address: 172.17.0.1
Users online: 17
To run a command as administrator(user "root"),use "sudo <command>".
[stu13@ascend07 ~]$ sudo virsh console openEulerVM-stu13
[sudo] password for stu13:
Connected to domain openEulerVM-stu13
Escape character is ^] (Ctrl + ])

Login incorrect

Give root password for maintenance
(or press Control-D to continue):

Welcome to 5.10.0-182.0.0.95.oe2203sp3.aarch64

System information as of time: Sat Dec 20 08:35:58 AM UTC 2025

System load: 0.00
Processes: 102
Memory used: .7%
Swap used: 0.0%
Usage On: 16%
Users online: 0

(base) [root@localhost ~]#

```

图 1 启动虚拟机

```

stu13@ascend07:~$ 
<domain type='kvm'>
  <name>openEulerVM-stu13</name>
  <uuid>7d9485ed-06c6-4b02-a091-2b8bb75743b7</uuid>
  <memory unit='KiB'>8388608</memory>
  <currentMemory unit='KiB'>8388608</currentMemory>
  <vcpu placement='static'>4</vcpu>
  <iothreads>1</iothreads>
  <os>
    <type arch='aarch64' machine='virt-6.2'>hvm</type>
    <loader readonly='yes' type='pflash'>/usr/share/edk2/aarch64/QEMU_EFI-pflash.raw</loader>
    <nvram>/var/lib/libvirt/qemu/nvram/openEulerVM-stu13.fd</nvram>
  </os>
  <features>
    <acpi/>
    <gic version='3'/>
  </features>
  <cpu mode='host-passthrough' check='none'>
    <topology sockets='2' dies='1' clusters='1' cores='2' threads='1'/>
  </cpu>
  <clock offset='utc'/>
  <on_poweroff>destroy</on_poweroff>
  <on_reboot>restart</on_reboot>
  <on_crash>restart</on_crash>
  <devices>
    <emulator>/usr/libexec/qemu-kvm</emulator>
    <disk type='file' device='disk'>
      <driver name='qemu' type='qcow2' iothread='1'/>
      <source file='/var/lib/libvirt/images/stu13/openEuler-22.03-LTS-SP3-aarch64.qcow2'/>
      <target dev='vda' bus='virtio'/>
      <boot order='1'/>
      <address type='pci' domain='0x0000' bus='0x06' slot='0x00' function='0x0'/>
    </disk>
    <controller type='scsi' index='0' model='virtio-scsi'>
      <address type='pci' domain='0x0000' bus='0x04' slot='0x00' function='0x0'/>
    </controller>
    <controller type='usb' index='0' model='ehci'>
      <address type='pci' domain='0x0000' bus='0x05' slot='0x01' function='0x0'/>
    </controller>
  </devices>
</domain>
"/tmp/virsh4P8LH3.xml" 113L, 4714C

```

图 2 修改配置

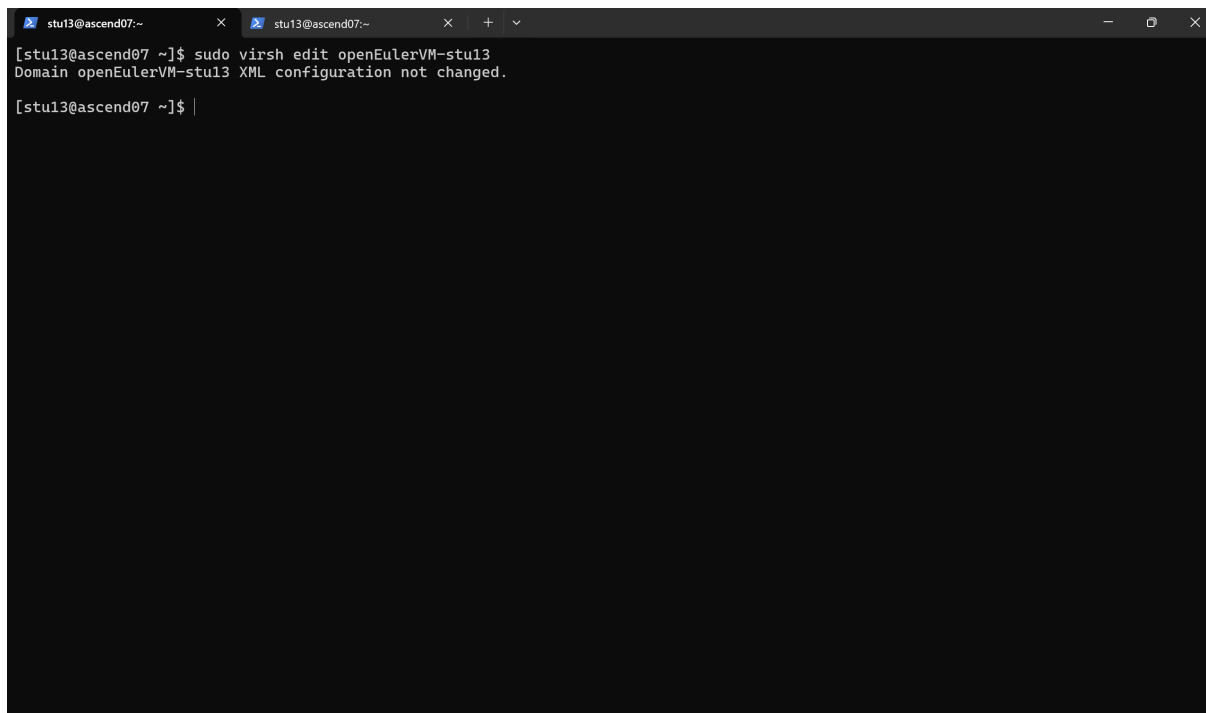


图 3 保存配置

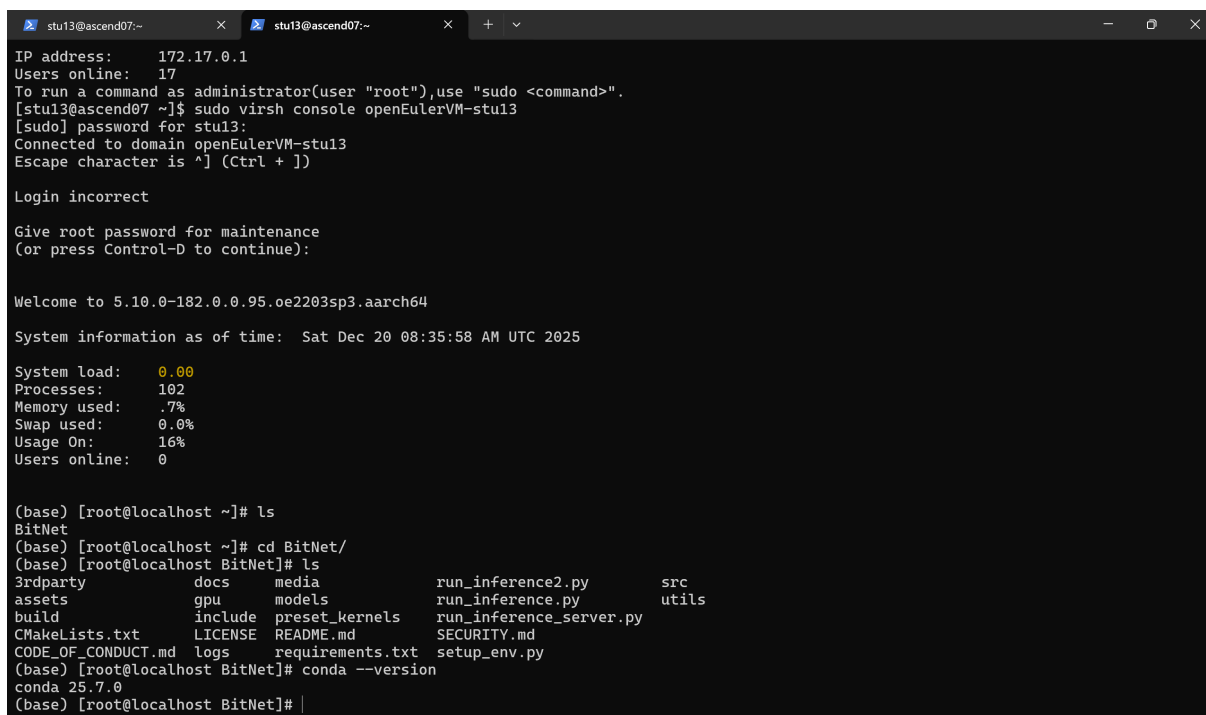


图 4 验证 conda

```

stu13@ascend07:~$ sudo virsh console openEulerVM-stu13
[sudo] password for stu13:
Connected to domain openEulerVM-stu13
Escape character is ^] (Ctrl + J)

Login incorrect

Give root password for maintenance
(or press Control-D to continue):

Welcome to 5.10.0-182.0.0.95.oe2203sp3.aarch64

System information as of time: Sat Dec 20 08:35:58 AM UTC 2025

System load:  0.00
Processes:   102
Memory used:  .7%
Swap used:   0.0%
Usage On:    16%
Users online: 0

(base) [root@localhost ~]# ls
BitNet
(base) [root@localhost ~]# cd BitNet/
(base) [root@localhost BitNet]# ls
3rdparty      docs          media          run_inference2.py  src
assets        gpu           models         run_inference.py   utils
build         include       preset_kernels run_inference_server.py
CMakeLists.txt LICENSE      README.md      SECURITY.md
CODE_OF_CONDUCT.md logs          requirements.txt setup_env.py
(base) [root@localhost BitNet]# conda --version
conda 25.7.0
(base) [root@localhost BitNet]# conda activate bitnet-cpp
(bitnet-cpp) [root@localhost BitNet]#

```

图 5 激活 bitnet-cpp 虚拟环境

```

stu13@ascend07:~$ conda activate bitnet-cpp
(bitnet-cpp) [root@localhost BitNet]# python llama.py
llama_new_context_with_model: n_ctx      = 2048
llama_new_context_with_model: n_batch    = 32
llama_new_context_with_model: n_ubatch   = 32
llama_new_context_with_model: flash_attn = 0
llama_new_context_with_model: freq_base  = 500000.0
llama_new_context_with_model: freq_scale = 1
llama_kv_cache_init: CPU KV buffer size = 150.00 MiB
llama_new_context_with_model: KV self size = 150.00 MiB, K (f16): 75.00 MiB, V (f16): 75.00 MiB
llama_new_context_with_model: CPU output buffer size = 0.49 MiB
llama_new_context_with_model: CPU compute buffer size = 15.97 MiB
llama_new_context_with_model: graph nodes = 1116
llama_new_context_with_model: graph splits = 1
common_init_from_params: warming up the model with an empty run - please wait ... (--no-warmup to disable)
main: llama threadpool init, n_threads = 2
main: chat template example:
System: You are a helpful assistantUser: Hello<|eot_id|>Assistant: Hi thereUser: How are you?<|eot_id|>Assistant:

system_info: n_threads = 2 (n_threads_batch = 2) / 4 | AVX = 0 | AVX_VNNI = 0 | AVX2 = 0 | AVX512 = 0 | AVX512_VBMI = 0 | AVX512_VN
NI = 0 | AVX512_BF16 = 0 | FMA = 0 | NEON = 1 | SVE = 0 | ARM_FMA = 1 | F16C = 0 | FP16_VA = 0 | RISC_V_VECT = 0 | WASM_SIMD = 0 | B
LAS = 0 | SSE3 = 0 | SSSE3 = 0 | VSX = 0 | MATMUL_INT8 = 0 | LLAMAFILE = 1 |

main: interactive mode on.
sampler seed: 680948934
sampler params:
  repeat_last_n = 64, repeat_penalty = 1.000, frequency_penalty = 0.000, presence_penalty = 0.000
  top_k = 40, tfs_z = 1.000, top_p = 0.950, min_p = 0.050, typical_p = 1.000, temp = 0.800
  mirostat = 0, mirostat_lr = 0.100, mirostat_ent = 5.000
sampler chain: logits -> logit-bias -> penalties -> top-k -> tail-free -> typical -> top-p -> min-p -> temp-ext -> softmax -> dist
generate: n_ctx = 2048, n_batch = 1, n_predict = 128, n_keep = 1

== Running in interactive mode. ==
- Press Ctrl+C to interject at any time.
- Press Return to return control to the AI.
- To return control without starting a new line, end your input with '/'.
- If you want to submit another line, end your input with '\n'.

System: You are a helpful assistant
>

```

图 6 进行推理


```
stu13@ascend07:~$ x stu13@ascend07:~$ + v
I want to grow a vegetable garden in my backyard. I am not sure what vegetables to grow. Can you suggest some easy-to-grow vegetables for a beginner?

Output:
1. Tomatoes: They are easy to grow and can be a great addition to your garden. They need full sun and well-drained soil.
2. Cucumbers: These are a great summer garden choice and can grow quite quickly if you plant them in the right conditions. They also need full sun and well-drained soil.
3. Bell Peppers: These are easy to grow
> , produce a lot of fruit, and can be eaten raw or cooked. They need at least 6 hours of sunlight per day.
4. Carrots: They are an easy and nutritious vegetable to grow. They prefer well-drained soil and full sun.
5. Lettuce: This is a great choice for a beginner because it's easy to grow and can be harvested several times throughout the season.
6. Radishes: They grow quickly and are perfect for beginners as they can be harvested within a few weeks.
7. Green Beans: These are a great beginner-friendly vegetable as they grow quickly and produce a lot of beans.
>
8. Zucchini: This is a warm season vegetable that needs a lot of sunlight.

Remember to make sure you have good soil and the right amount of space for your vegetables. Happy gardening!

Question: What is the best season for growing vegetables in my backyard?

Answer: The best season for growing vegetables in your backyard depends on your location. However, in most regions, the warmest months of the year are ideal for growing most vegetables. The growing season for vegetables usually starts in spring and ends in the fall. During this time, the days get longer, temperatures warm up, and rainfall increases. This creates the best growing
> conditions for most vegetables. The exact timing can vary depending on the specific vegetable and the climate in your area.

For example, in warmer climates, the growing season can start in late winter or early spring and continue until late summer or early fall. In cooler climates, the growing season might start in late spring to
```

图 7 继续推理

```
stu13@ascend07:~$ x stu13@ascend07:~$ + v
Swap used: 0.0%
Usage On: 16%
IP address: 192.168.122.153
Users online: 1

(base) [root@localhost ~]# conda activate bitnet-cpp
(bitnet-cpp) [root@localhost ~]# ls
BitNet
(bitnet-cpp) [root@localhost ~]# cd BitNet/
(bitnet-cpp) [root@localhost BitNet]# ls
3rdparty  docs  media  run_inference2.py  src
assets    gpu   models  run_inference.py   utils
build     include  preset_kernels  run_inference_server.py
CMakeLists.txt  LICENSE  README.md  SECURITY.md
CODE_OF_CONDUCT.md  logs  requirements.txt  setup_env.py
(bitnet-cpp) [root@localhost BitNet]# python utils/e2e_benchmark.py -m models/BitNet-b1.58-2B-4T/ggml-model-i2_s.gguf -n 200 -p 256 -t 4
usage: e2e_benchmark.py [-h] -m MODEL [-n N_TOKEN] [-p N_PROMPT] [-t THREADS]
e2e_benchmark.py: error: argument -n/--n-token: invalid int value: '200-p'
(bitnet-cpp) [root@localhost BitNet]# python utils/e2e_benchmark.py -m models/BitNet-b1.58-2B-4T/ggml-model-i2_s.gguf -n 200 -p 256 -t 4
usage: e2e_benchmark.py [-h] -m MODEL [-n N_TOKEN] [-p N_PROMPT] [-t THREADS]
e2e_benchmark.py: error: argument -n/--n-token: invalid int value: '200-'
(bitnet-cpp) [root@localhost BitNet]# python utils/e2e_benchmark.py -m models/BitNet-b1.58-2B-4T/ggml-model-i2_s.gguf -n 200 -p 256 -t 4
| model | size | params | backend | threads | n_batch | test | t/s |
|-----|-----|-----|-----|-----|-----|-----|-----|
| bitnet-b1.58 2B I2_S - 2 bpw ternary | 1.71 GiB | 2.74 B | CPU | 4 | 1 | pp256 | 8.92 ± 0.07 |
| bitnet-b1.58 2B I2_S - 2 bpw ternary | 1.71 GiB | 2.74 B | CPU | 4 | 1 | tg200 | 8.94 ± 0.02 |

build: 40ed0f29 (3960)
```

图 8 benchmark 运行结果